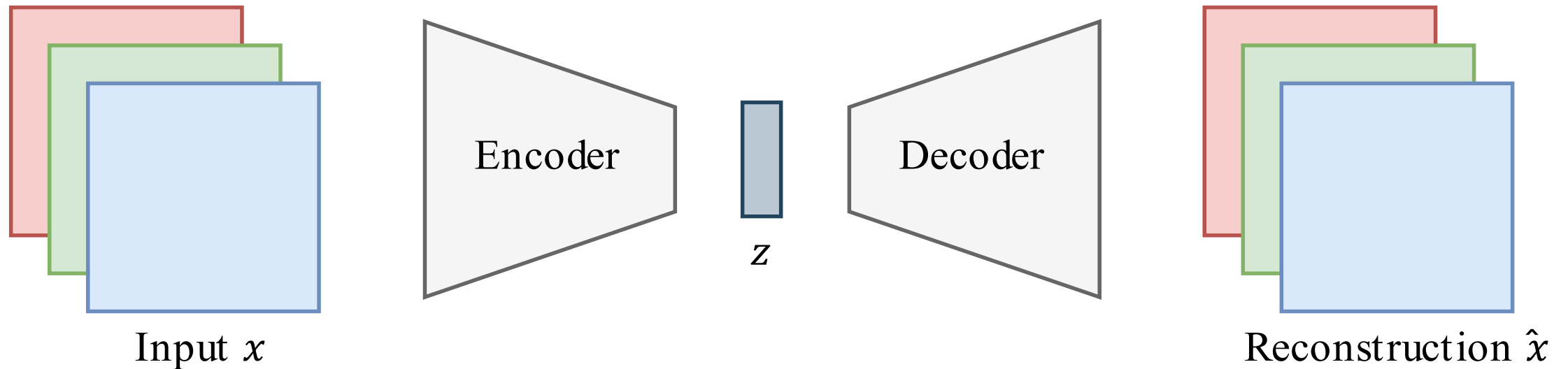


Introduction to Autoencoders and GANs

Reminder on Encoder-Decoder Networks

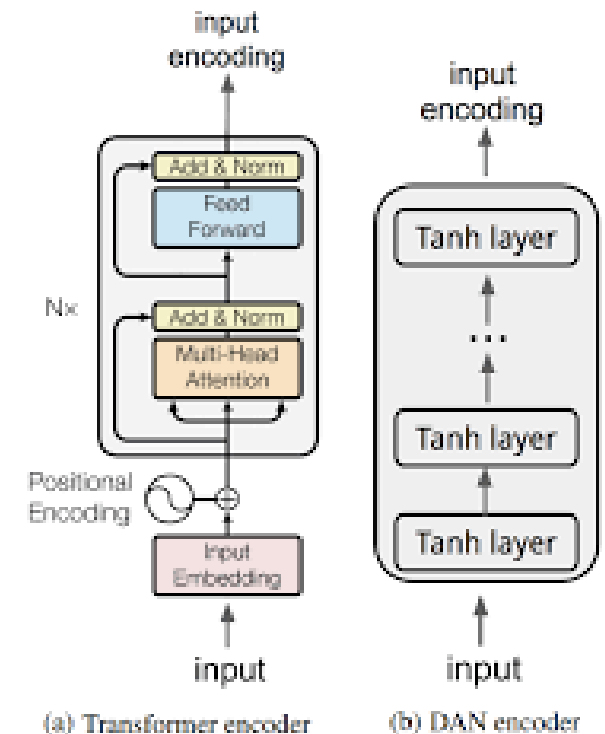
- **Encoder:** processes the input sequence to produce an **encoded representation**.
- **Decoder:** uses the **encoded representation** to generate an output sequence.



Reminder on Encoder-Decoder Networks

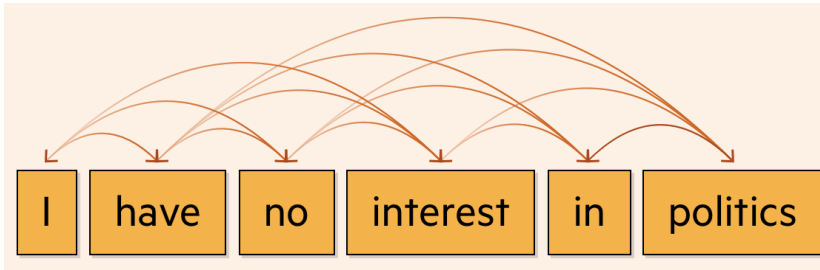
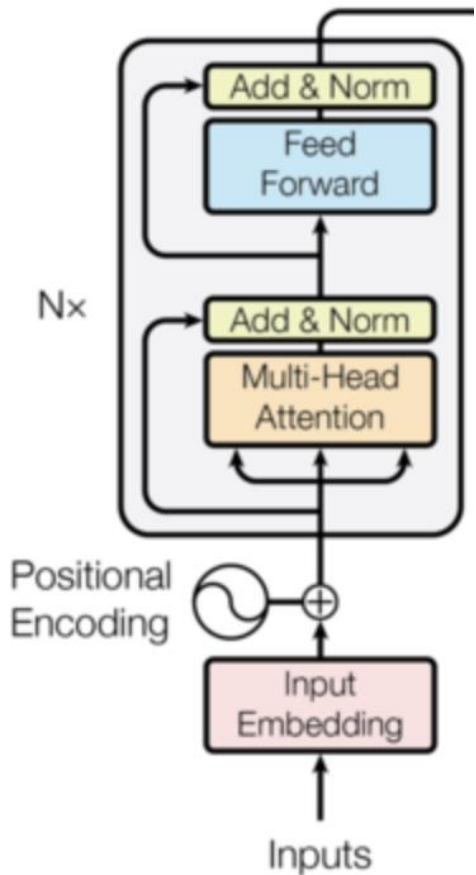
- **Encoder:** processes the input sequence to produce an **encoded representation**.
- **Decoder:** uses the **encoded representation** to generate an output sequence.

Transformers utilise this encoder-decoder network for the NLP tasks – but **crucially add ‘self-attention’ layers** to add higher weighting to the most important parts of the sequence to reduce ‘information bottleneck’ and produce more relevant outputs.



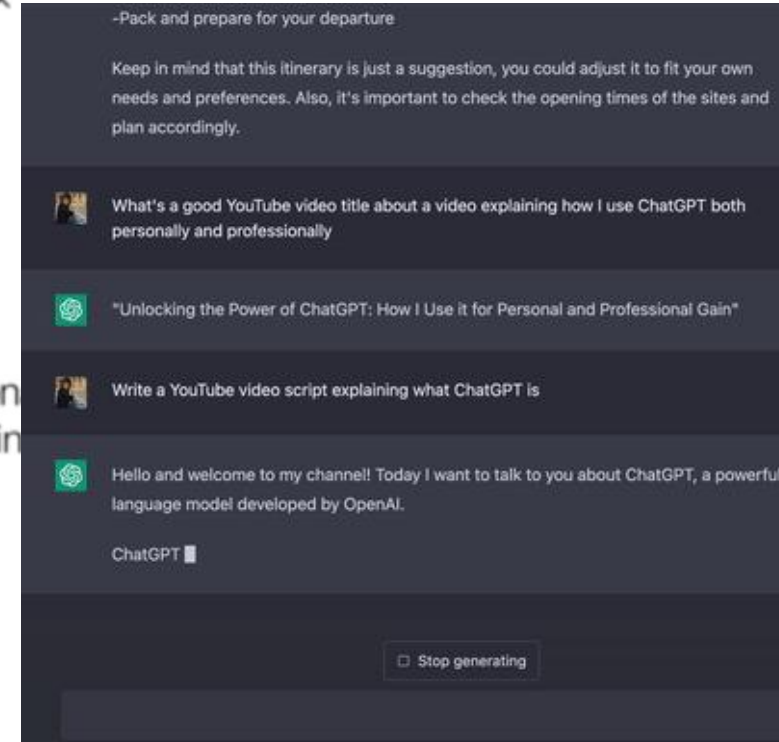
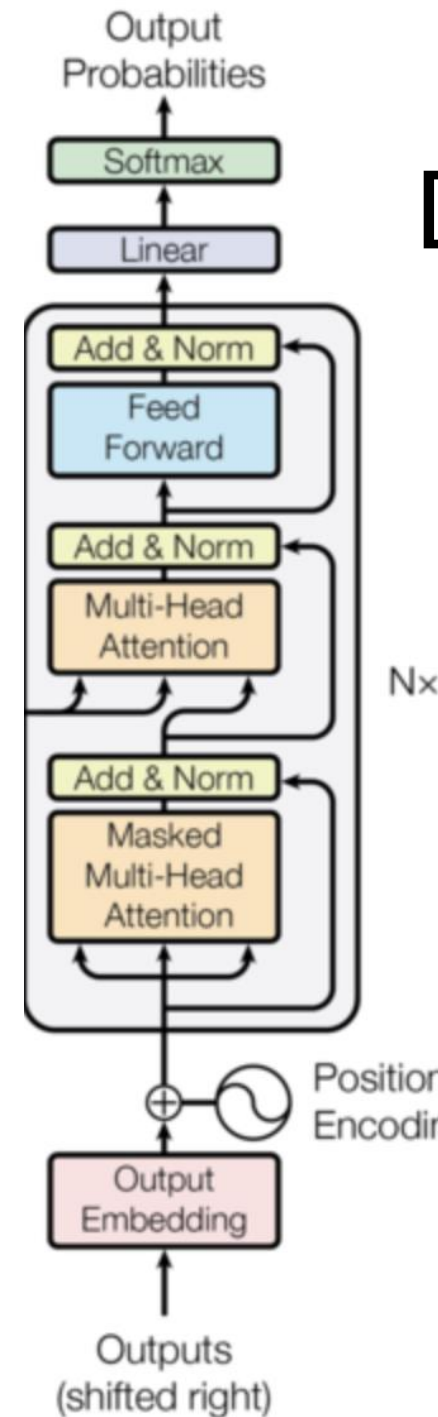
Encoder

Encoders (BERT) extract essential information from an input sequence and learn embeddings = text understanding

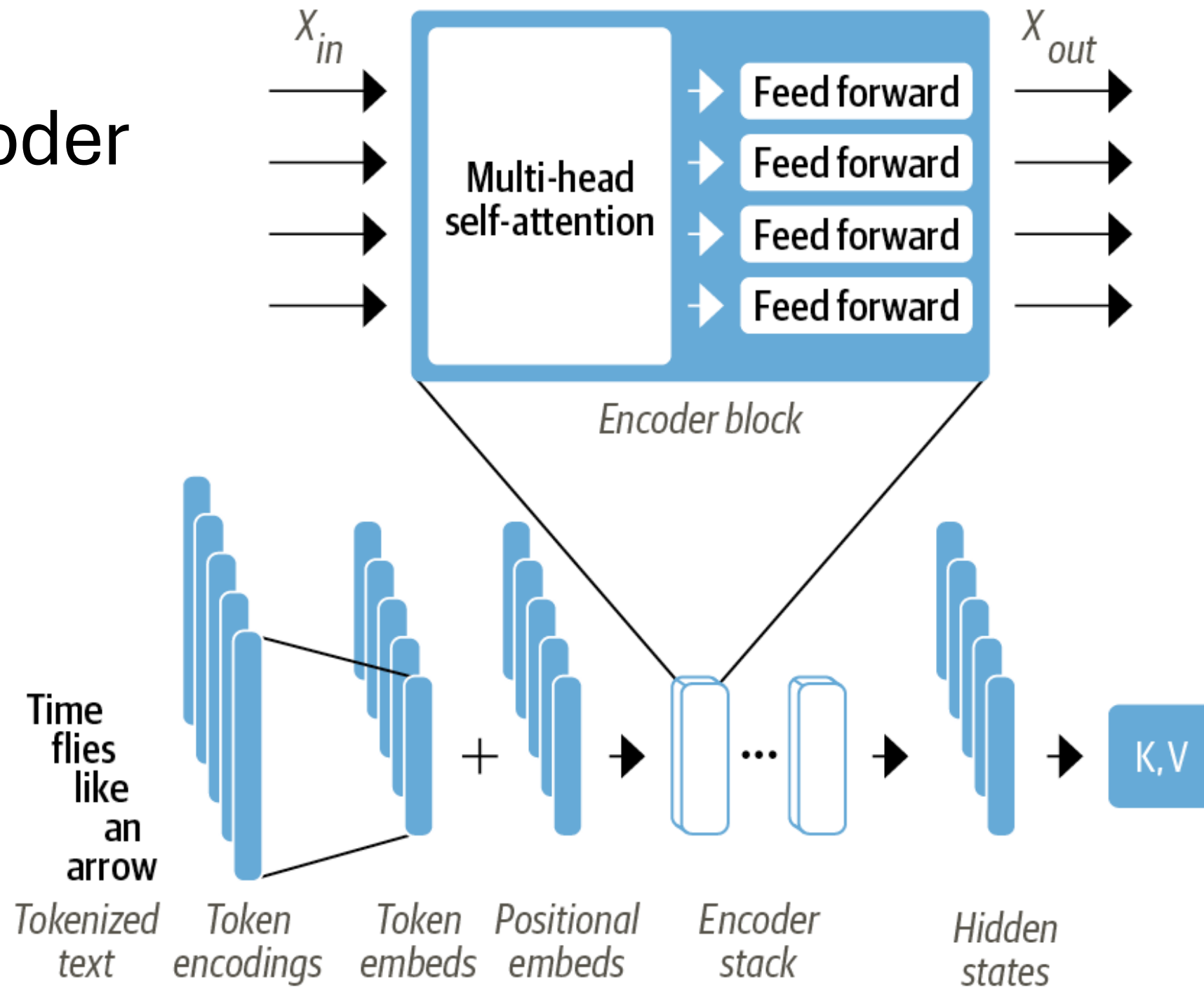


Decoder

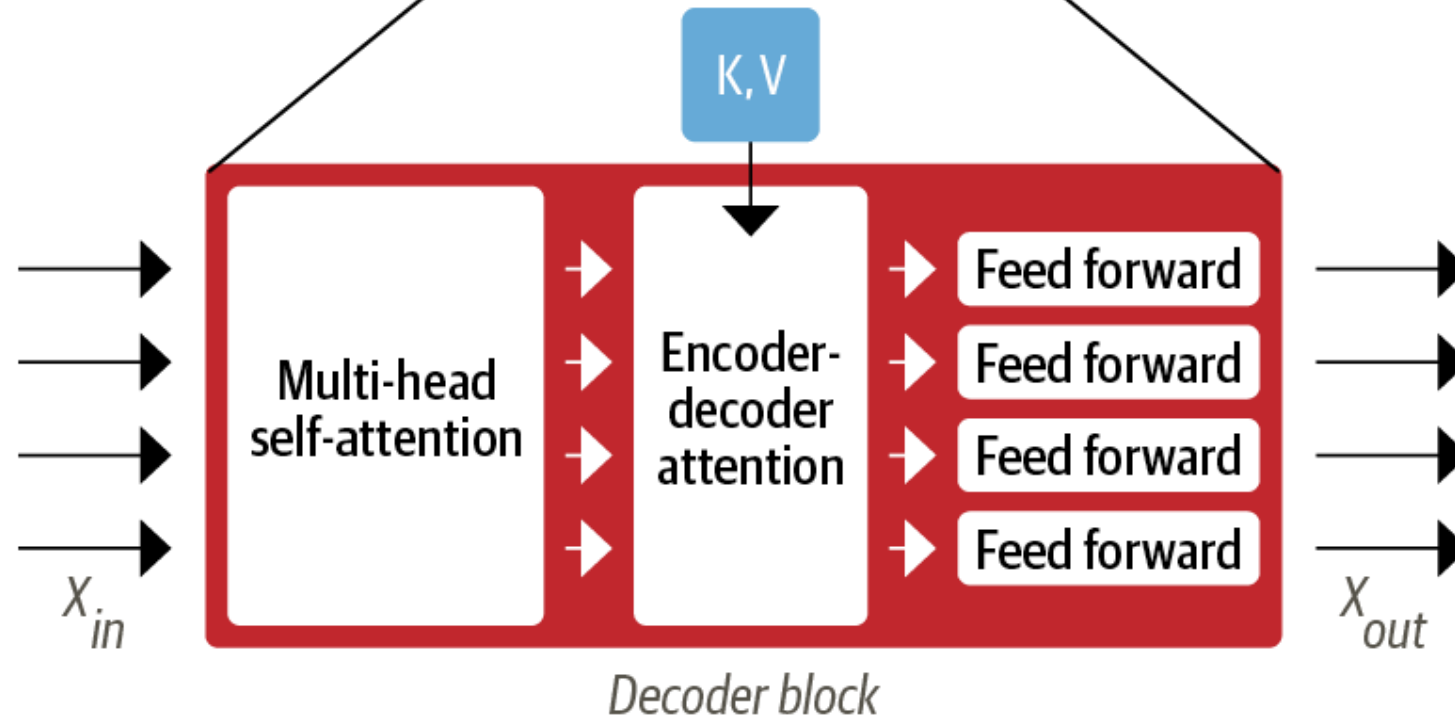
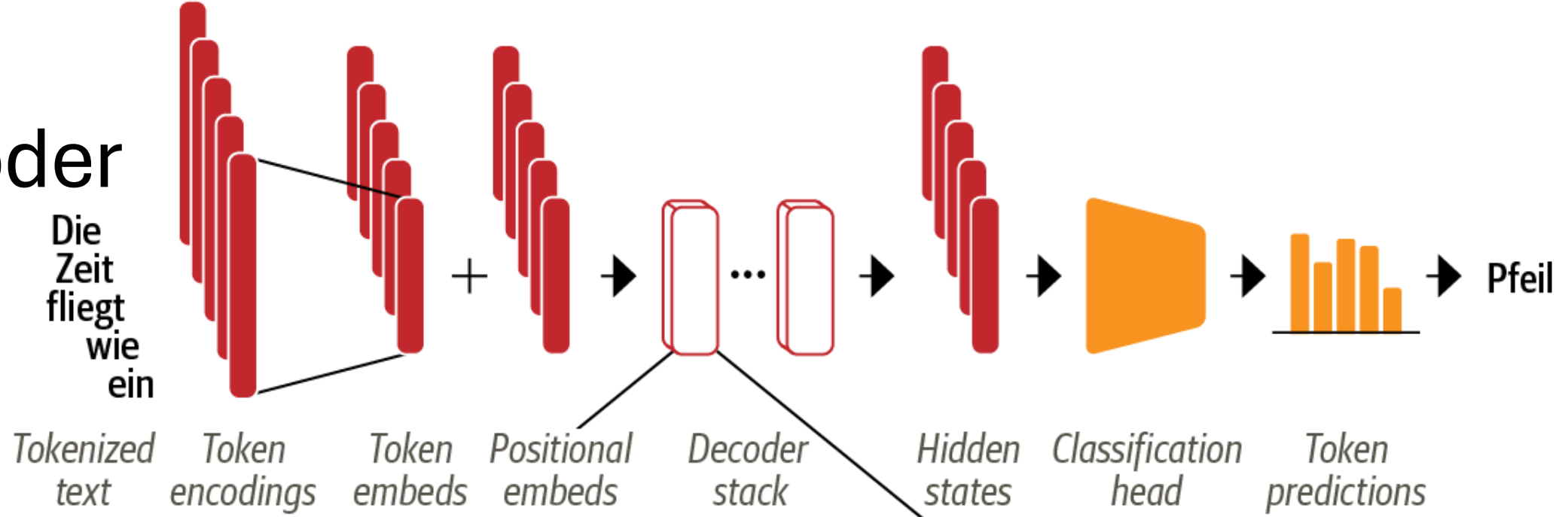
Decoders (GPT) are designed to predict new word sequences based on context provided (prompts) = text generation



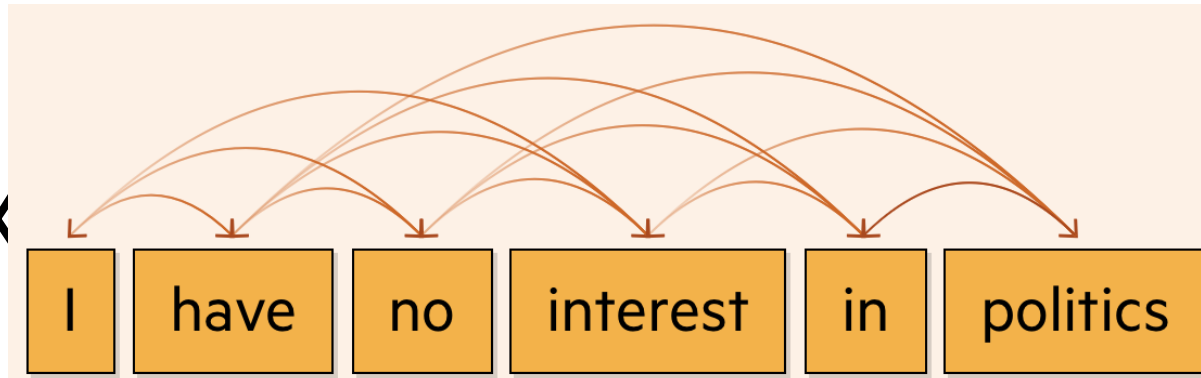
Encoder



Decoder



Encoder / Decoder – nex



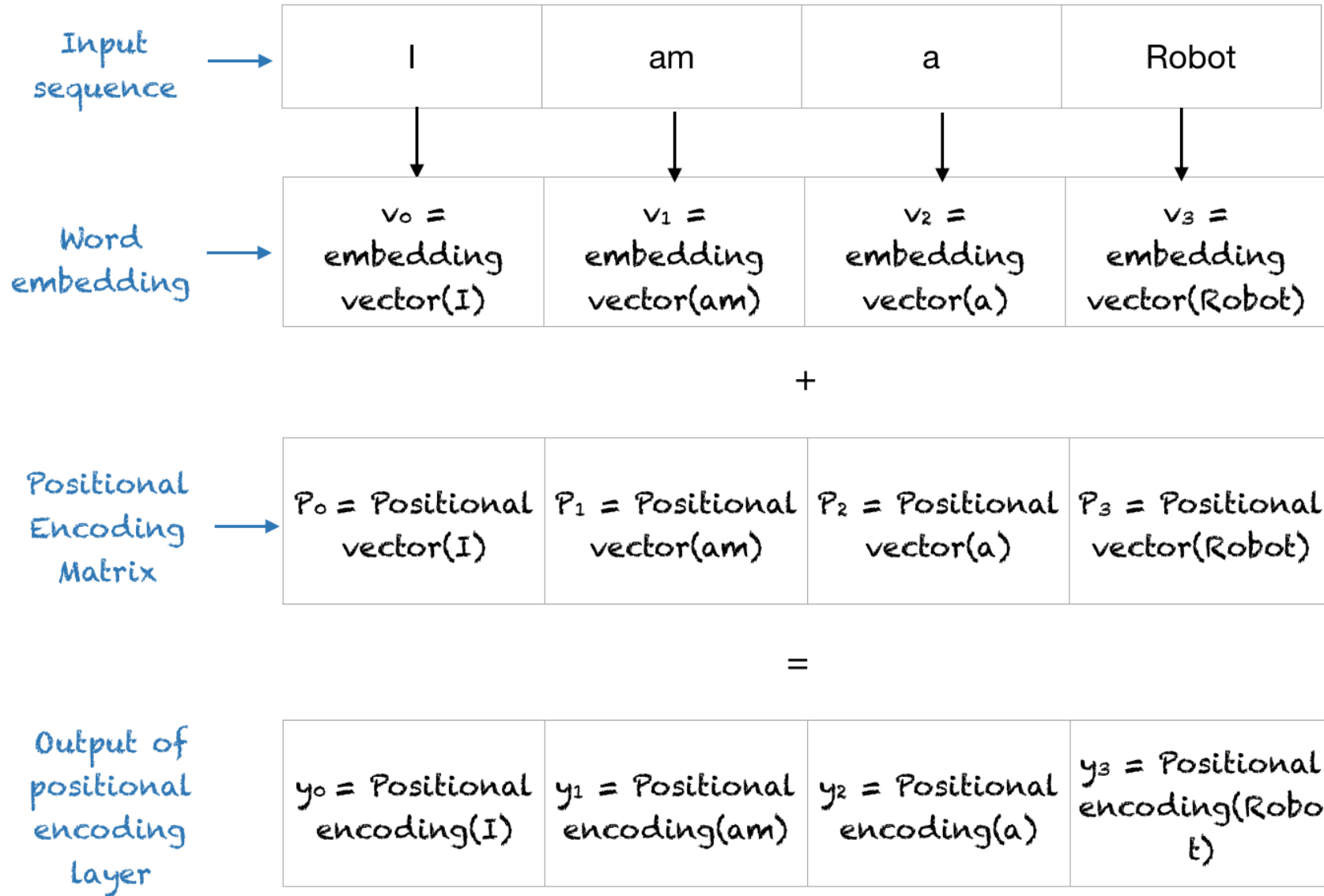
- GPT and Microsoft Copilot are both **decoder-only models**. This means that given words only have relationships with all words that have come before them (not in-front of them)
 - This is known as causal self-attention
 - Text is generated autoregressively
 - Decoder-only models are optimised for producing language
- Whereas bidirectional **encoder-based models** (e.g. BERT) – apply the full self-attention concept – with a given word token having relationships with all given words (past and future) in the sequence.
 - These bidirectional models are trained for understanding text
 - However, encoder models can't answer questions or generate responses to prompts and questions you ask it (like GPT can!).

Encoder example: Positional Encoding

Sequence		Index of token, k	Positional Encoding Matrix with $d=4$, $n=100$				
			$i=0$	$i=0$	$i=1$	$i=1$	
I	→	0	→	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	→	1	→	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	→	2	→	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	→	3	→	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

Encoder example: embedding + position



Encoder example: output – numbers!

```
Final output:  tf.Tensor(  
[[[ 0.03060279  0.03302038 -0.00112823  0.04488726  0.00671769 ← I  
    0.09401638]  
 [ 0.00138248  0.05229304 -0.08077629  0.07858343 -0.04831553 ← am  
   -0.02122493]  
 [ 0.0164414   0.01181945  0.03451525  0.02141005  0.03021857 ← a  
   -0.03256821]  
 [-0.05411192  0.00862638 -0.06338423 -0.04380698 -0.00718099 ← robot  
   -0.00852975]  
 [ 0.0169894  -0.00668144  0.01477278  0.00132753  0.00488833  
   -0.07528981]]]
```

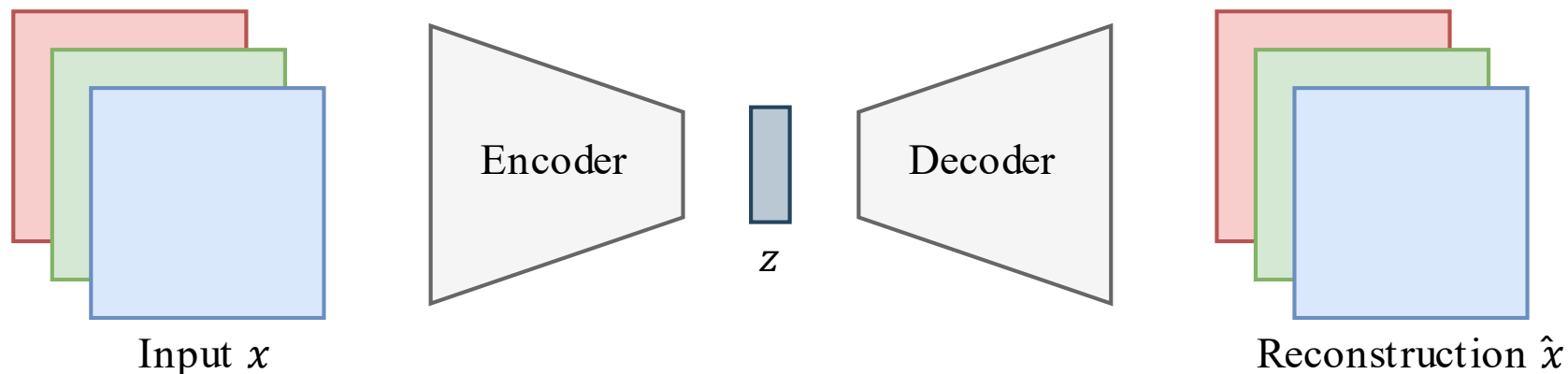
Decoder – requires **context** to reconstruct natural language!

```
Final output: tf.Tensor(  
[[[ 0.03060279  0.03302038 -0.00112823  0.04488726  0.00671769 → I  
    0.09401638]  
 [ 0.00138248  0.05229304 -0.08077629  0.07858343 -0.04831553 → am  
    -0.02122493]  
 [ 0.0164414   0.01181945  0.03451525  0.02141005  0.03021857 → a  
    -0.03256821]  
 [-0.05411192  0.00862638 -0.06338423 -0.04380698 -0.00718099 → robot  
    -0.00852975]  
 [ 0.0169894  -0.00668144  0.01477278  0.00132753  0.00488833  
    -0.07528981]]]
```

Autoencoders

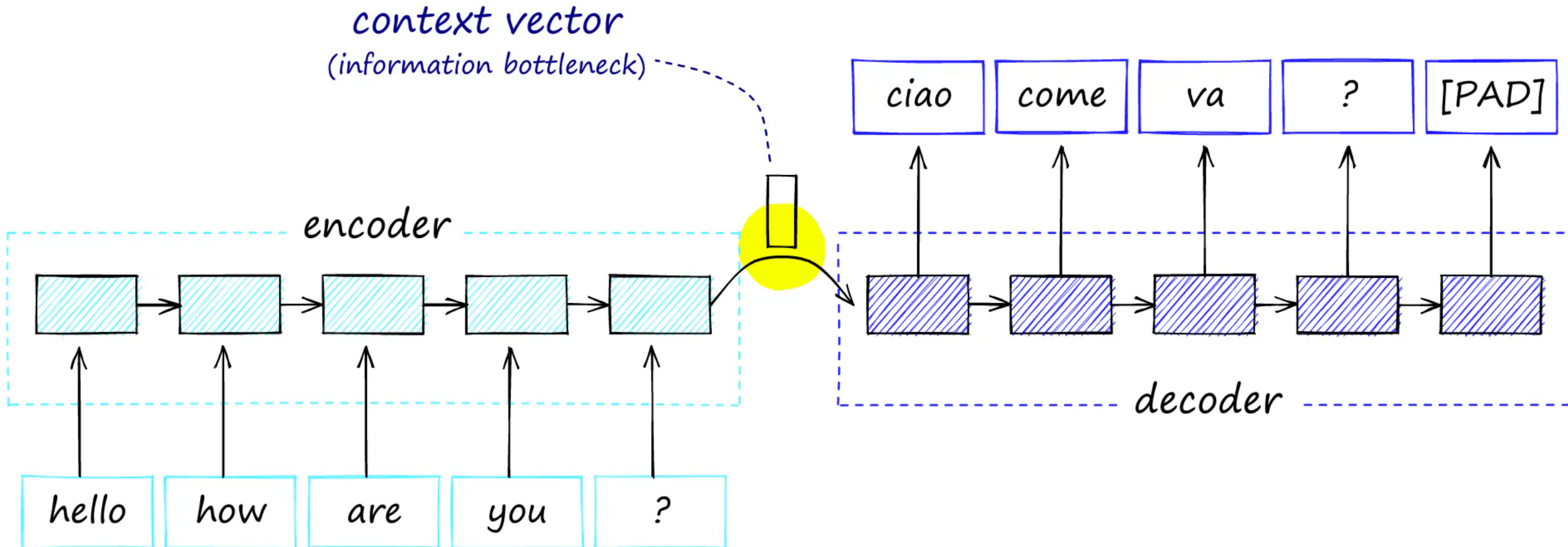
Autoencoders

- Auto-encoders are networks that **automate** the approach of encoding – which is to **learn efficient representations of data**.
- This automation is useful with **unsupervised learning**, as it doesn't require guidance/supervision.
- Autoencoders are designed to learn **compressed** representations of data (encoding) and then reconstruct the data from these representations (decoding).



Information bottleneck

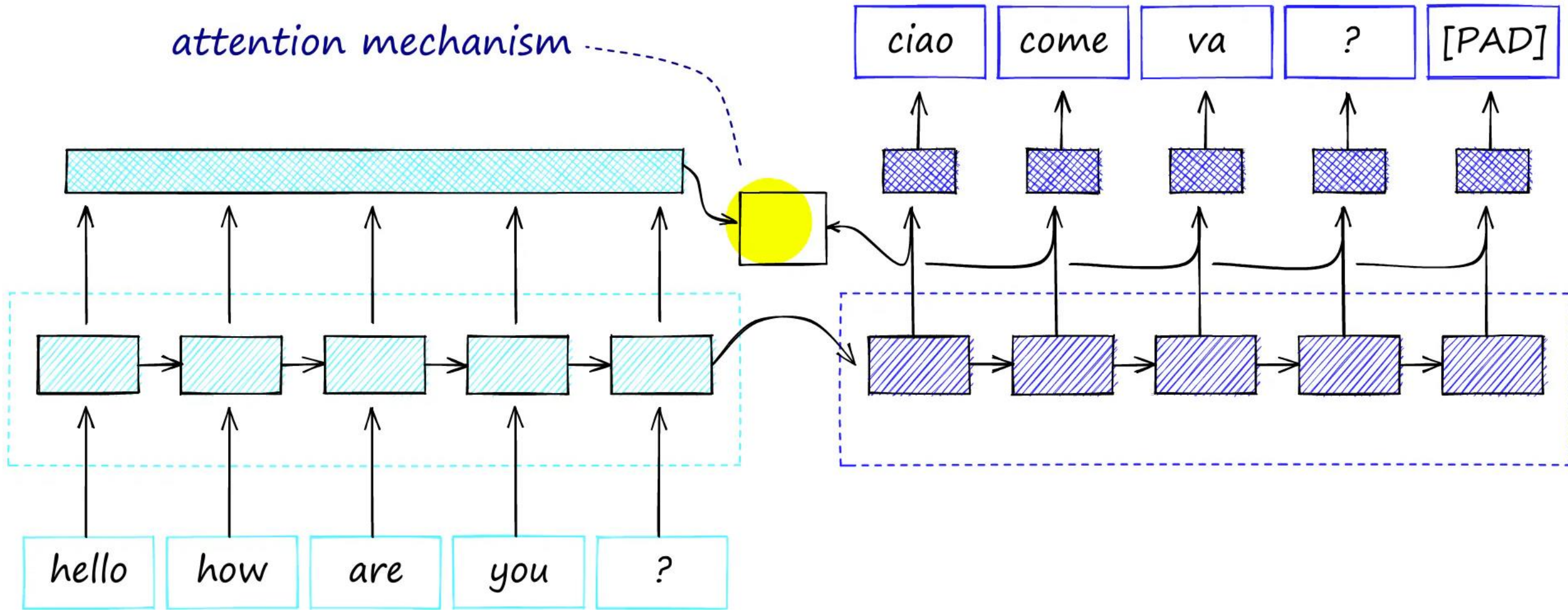
- All information must be passed through this context vector = information bottleneck.



Machine Translation: Attention mechanism

- Attention mechanism provides another route for information to pass through. Doesn't overwhelm due to the focus on most relevant information.

attention mechanism

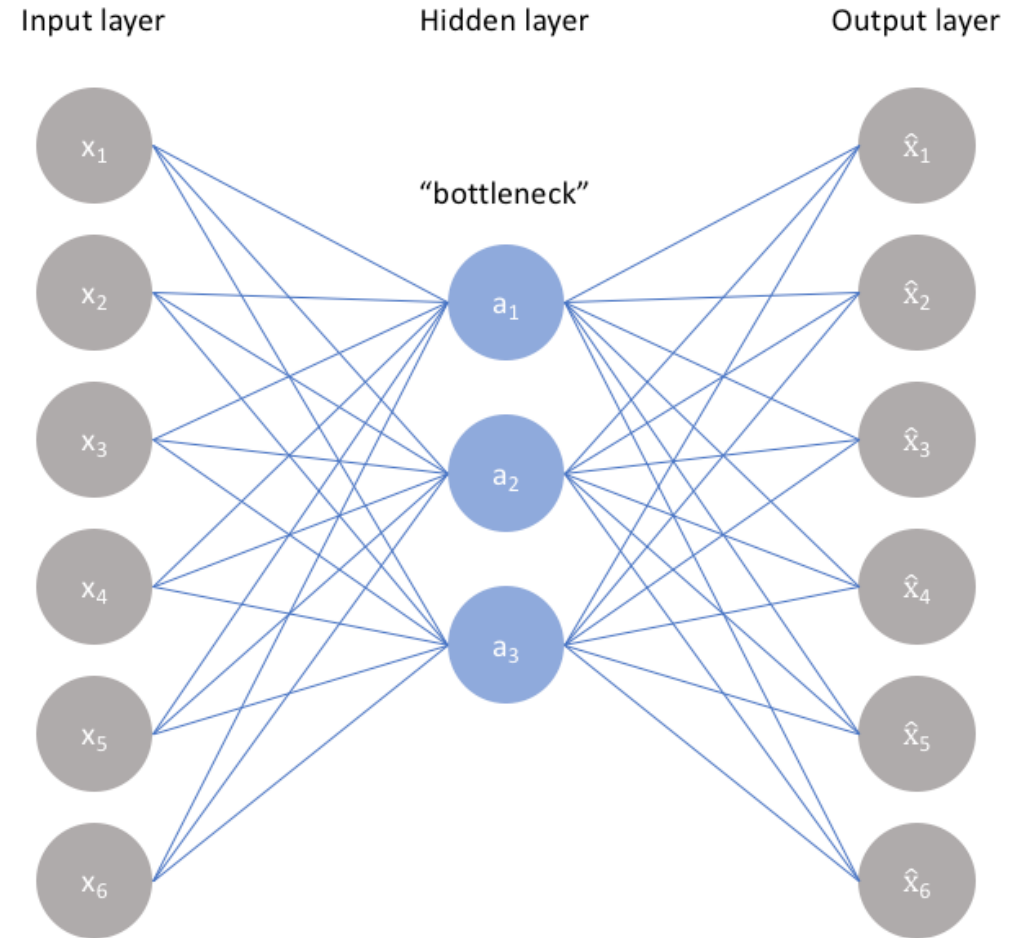


Autoencoders for dimensionality reduction

- Autoencoders are artificial neural networks capable of learning dense representations of the input data, called **latent representations or codings**, in an **unsupervised manner** (i.e., the training set is unlabeled).
- These codings typically have a **much lower dimensionality** than the input data, making autoencoders useful for dimensionality reduction, especially for visualization purposes.
- Autoencoders also act as **feature detectors**, and they can be used for unsupervised pretraining of deep neural networks.
- Finally, some autoencoders are **generative models**: they are capable of randomly generating new data that looks very similar to the training data.

Autoencoder Components

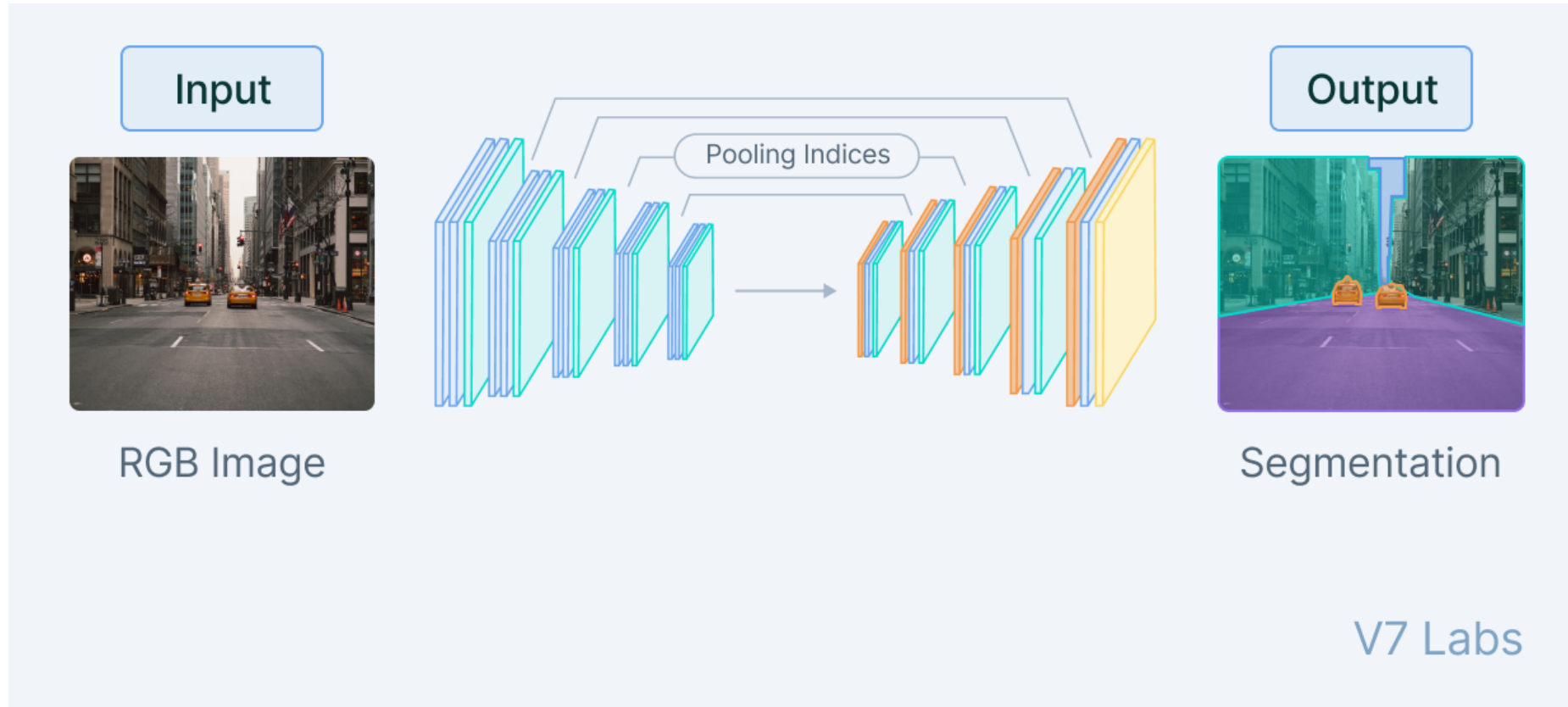
- Encoder: Maps the input data to a lower-dimensional latent space.
- Latent Space: The compressed representation of the input data.
- Decoder: Reconstructs the input data from the latent space representation.



Types of Autoencoders

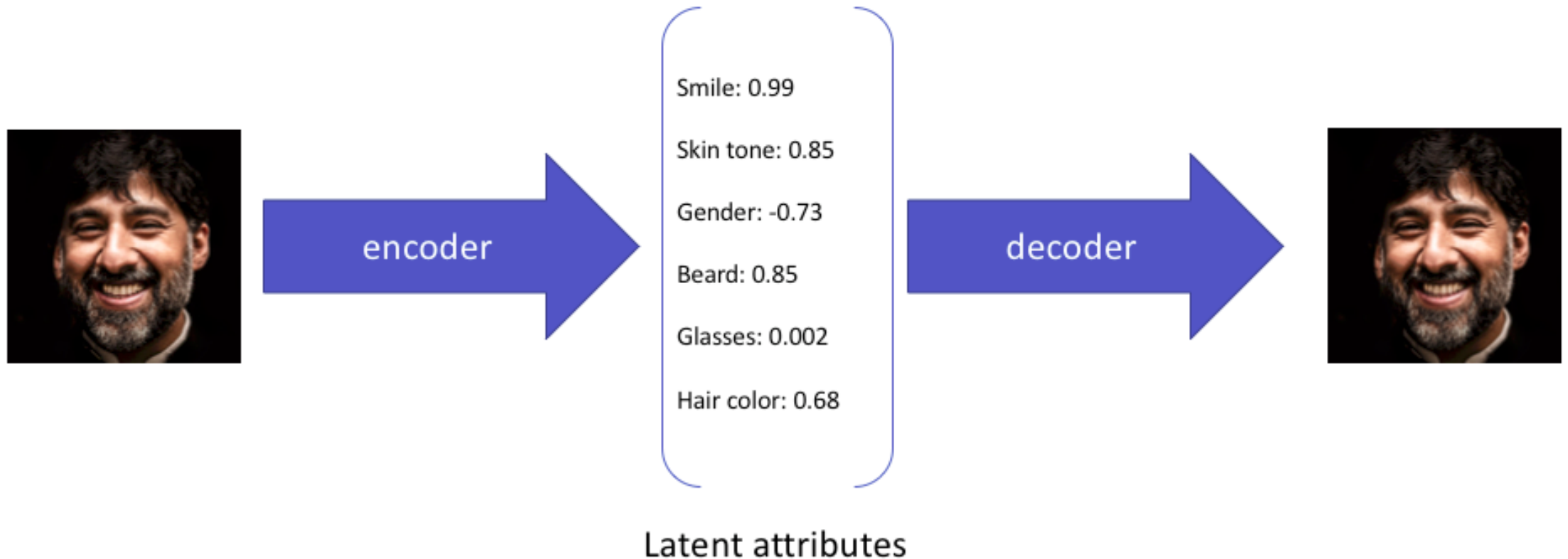
- **Basic Autoencoder** – encoder reduces dimensionality, then decoder attempts to reconstruct from reduced dim.
- **Denoising Autoencoder** – encoder compresses despite noise, then decoder attempts to reconstruct without noise
- **Variational Autoencoder (VAEs)** – doesn't learn fixed latent representations – instead generates probability distributions – useful for generating media (images, audio etc)
- **Convolutional Autoencoder (CAEs)** – encoder adapts convolutional layers found in CNNs to learn / extract 'features'. Decoder can then apply filters to the image, or up-sampling to restore the image.

Convolutional encoder-decoder for images



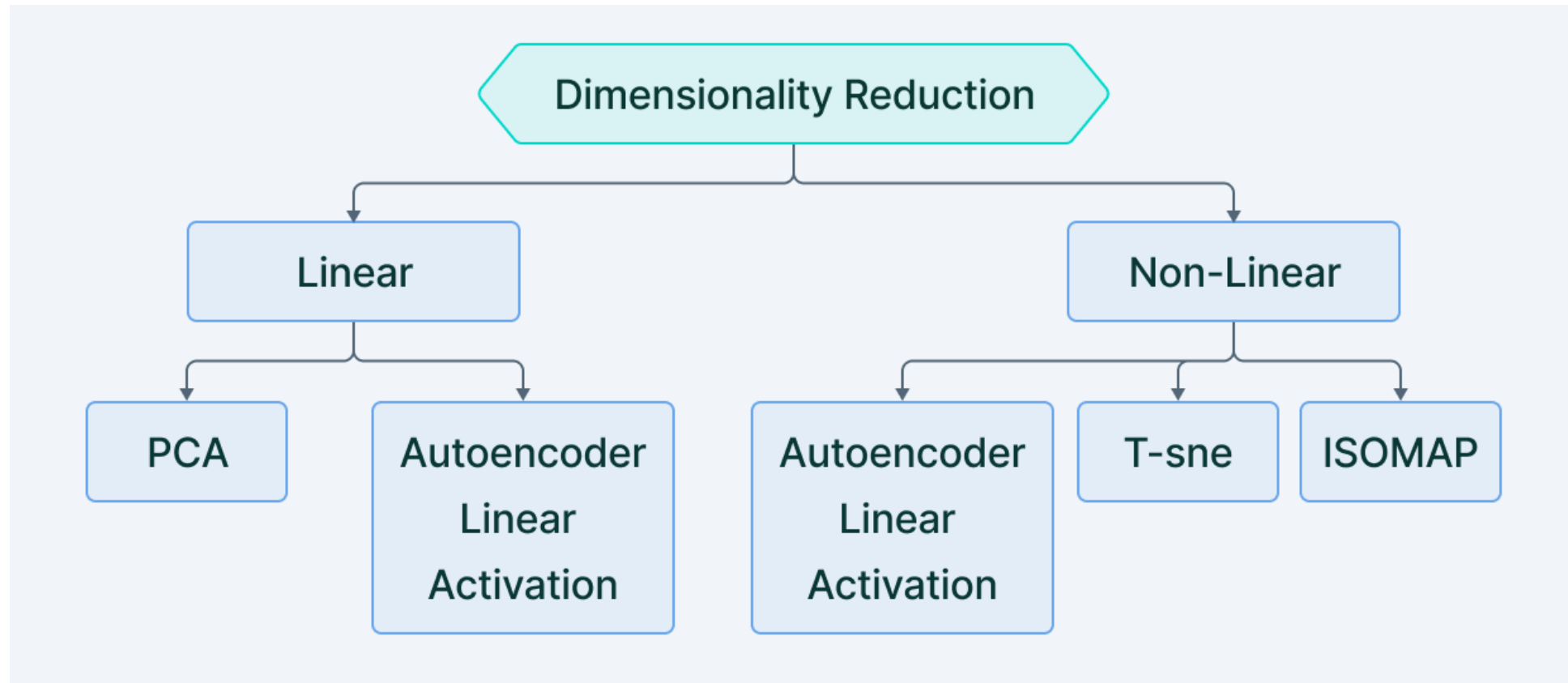
Latent space = learned attributes

- The latent space formed after training the model is not necessarily continuous and might not be easy to interpolate.



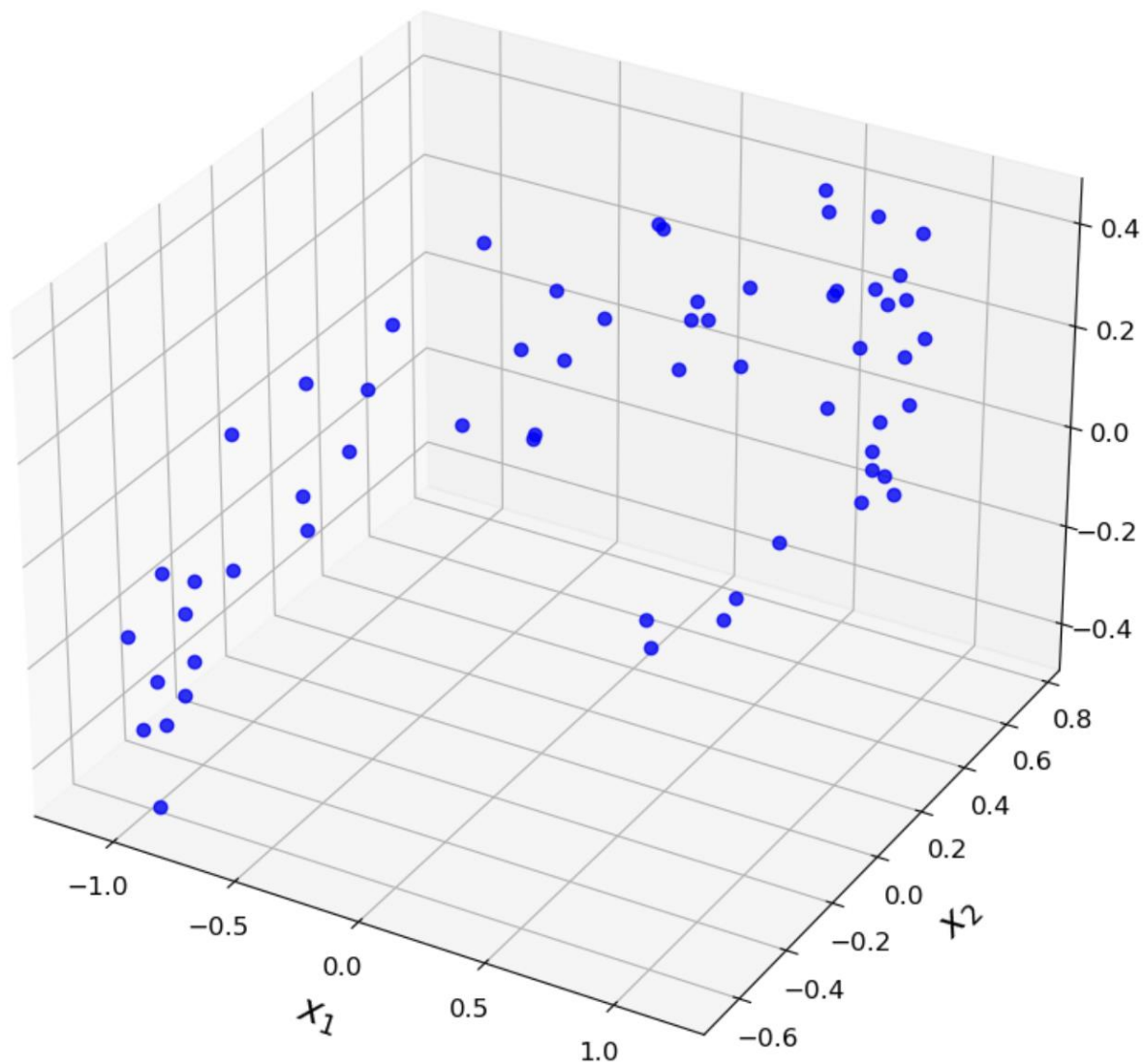
Autoencoders Application: dimensionality reduction

- PCA can only build linear relationships
- ANNs specialise in **non-linear** relationships

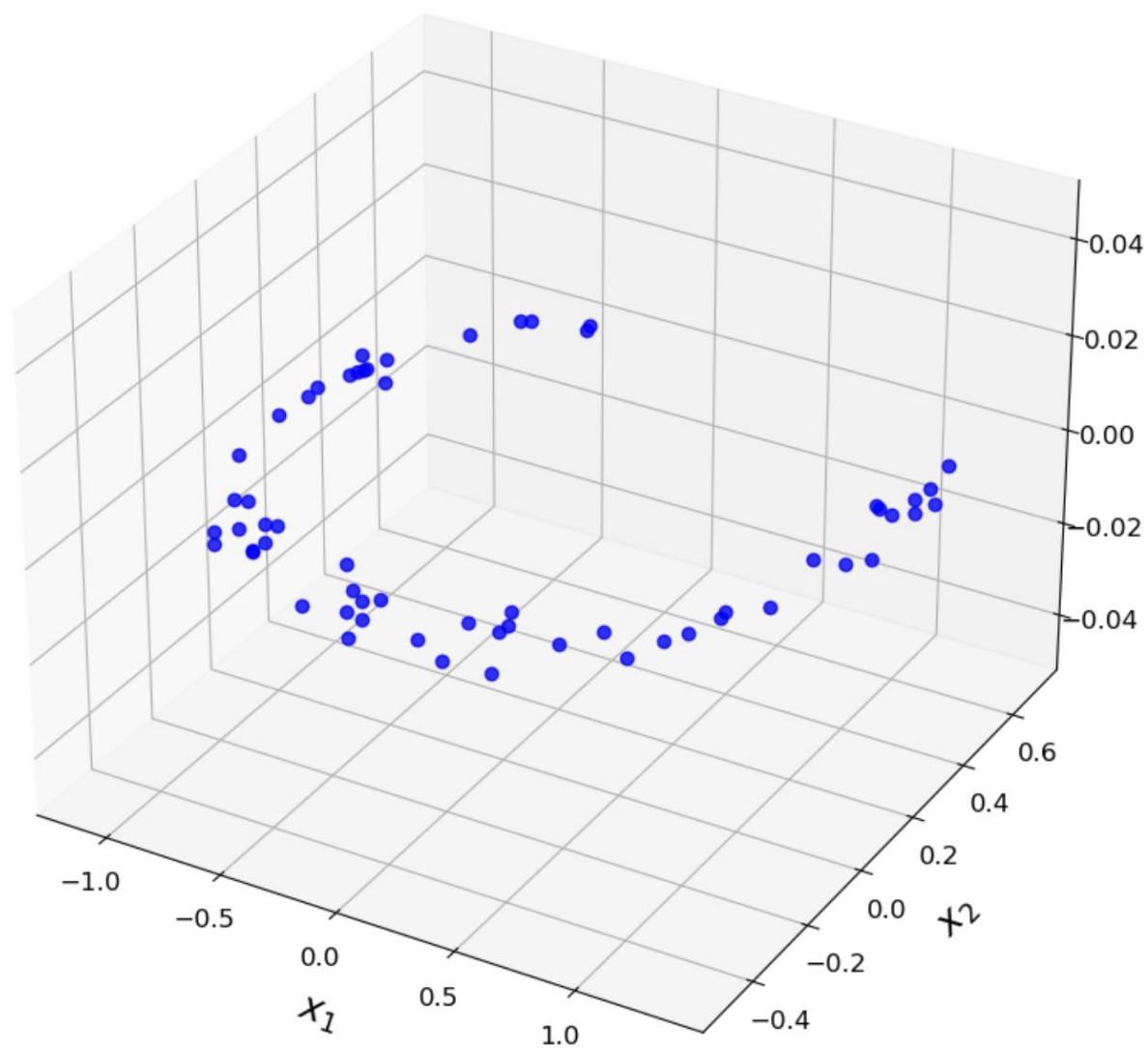


PCA

3D Dataset

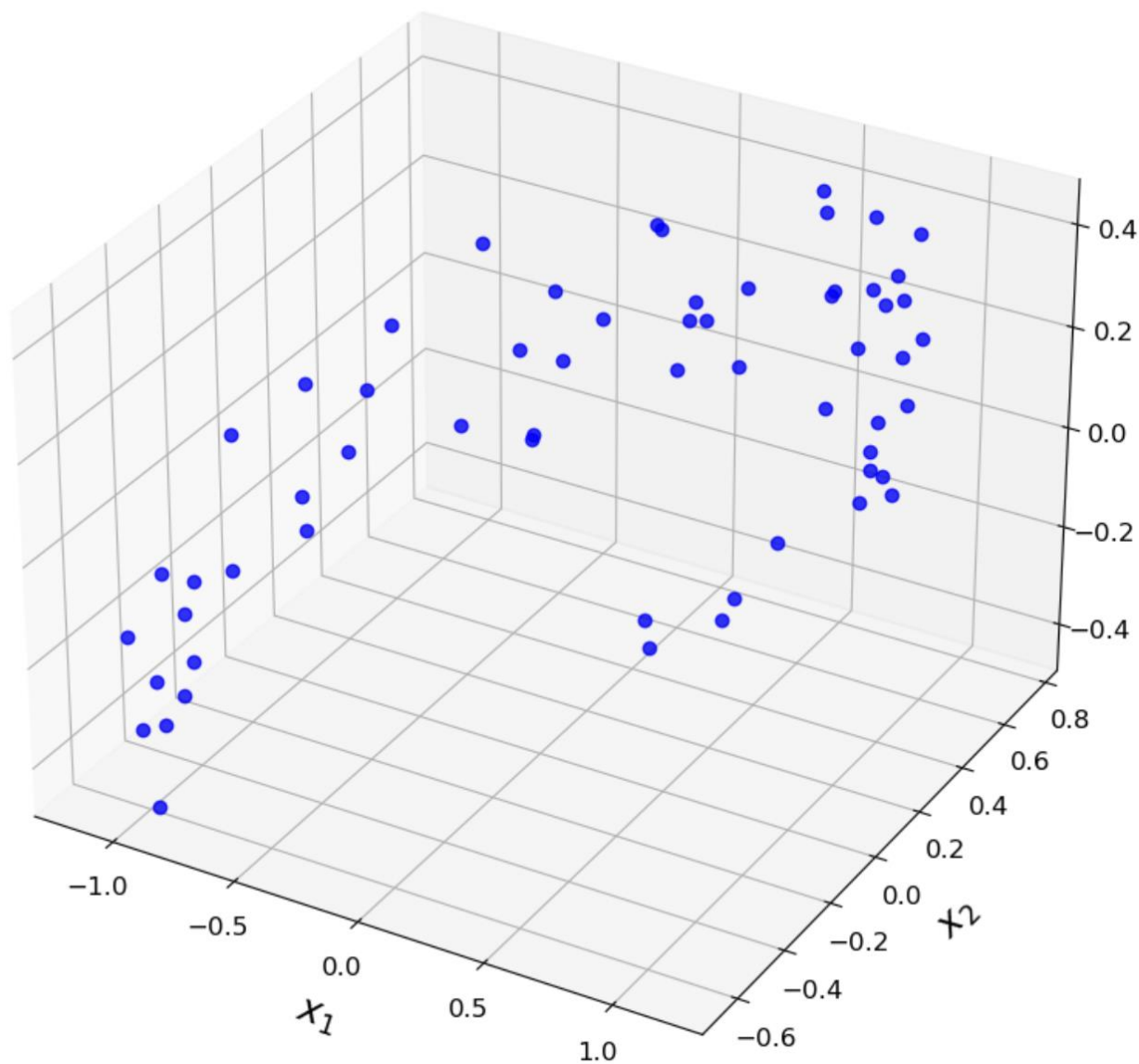


2D Dataset drawn in 3D space

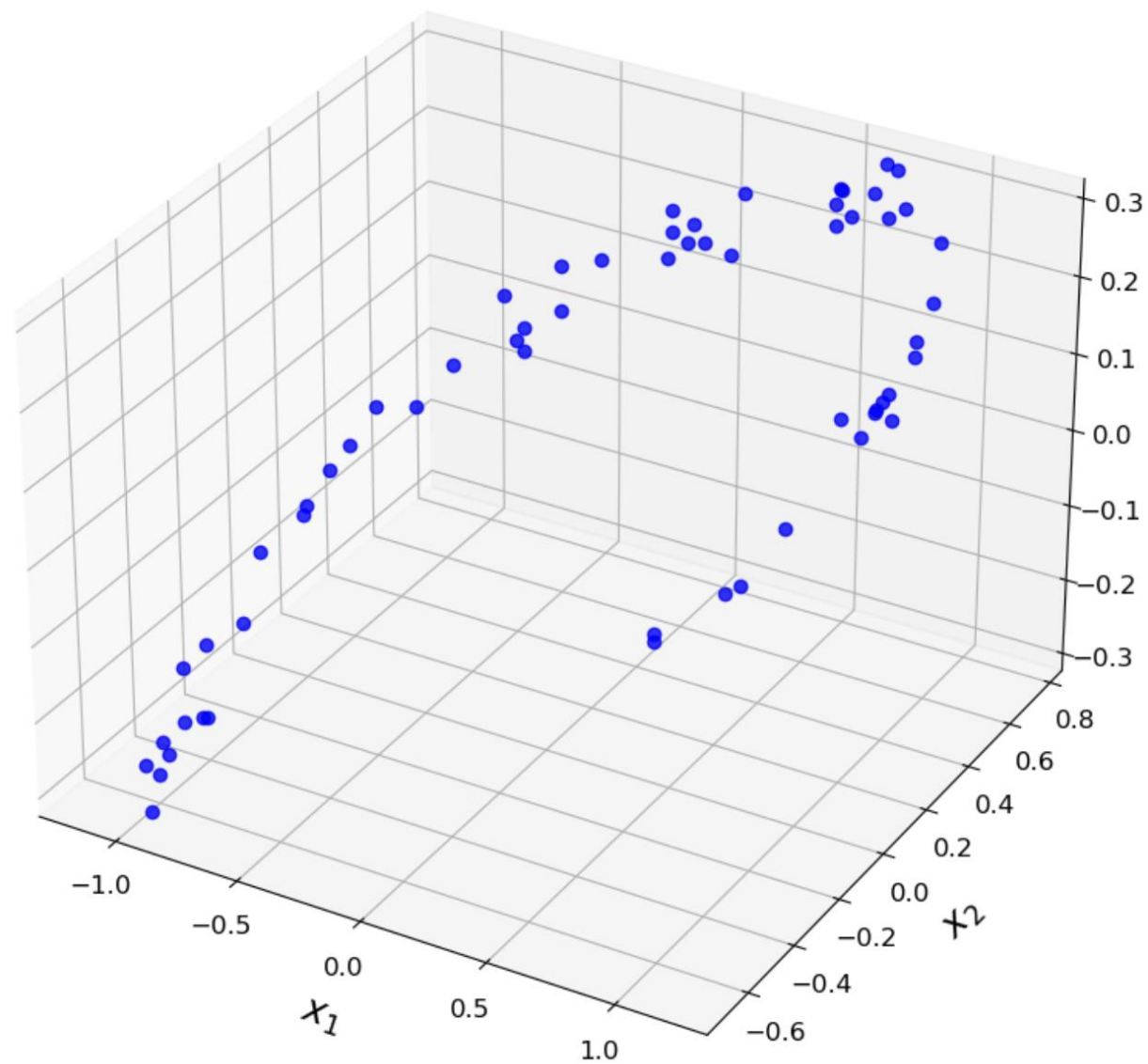


PCA

Original 3D Dataset

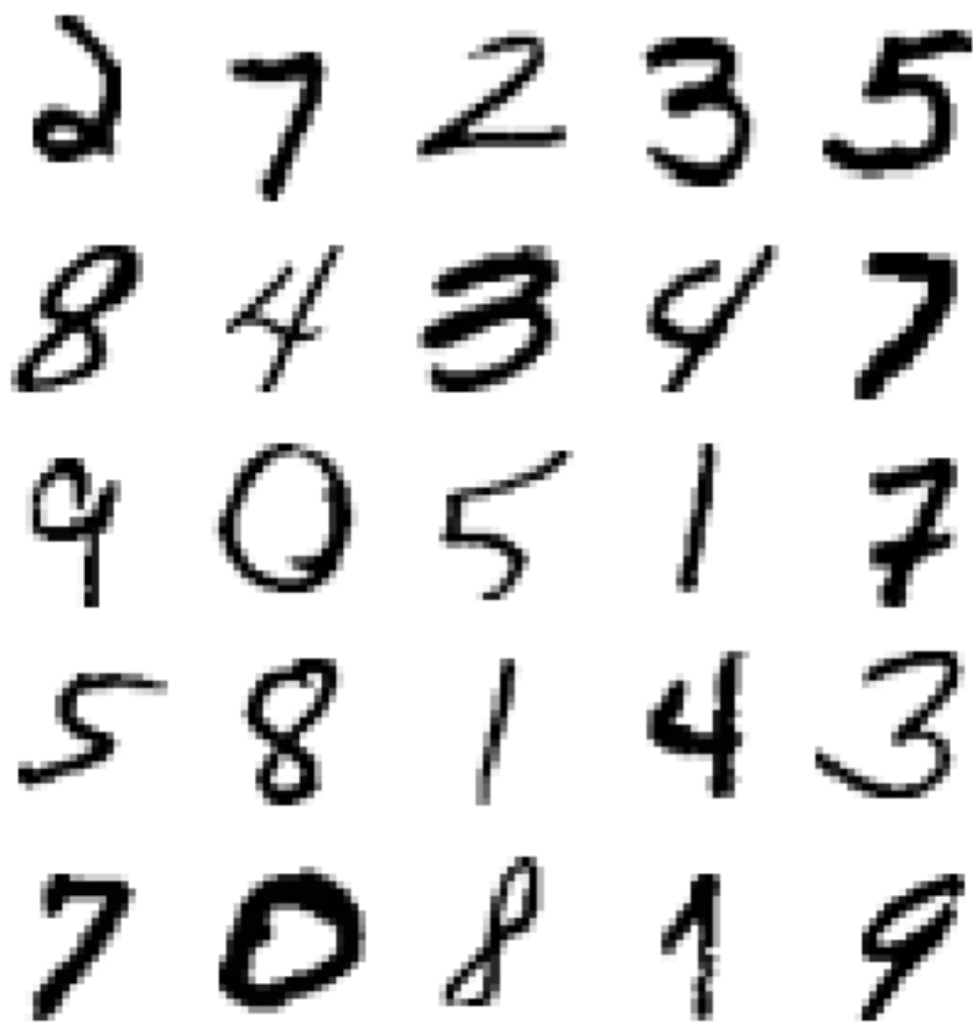


2D back to 3D Dataset (Inverted)



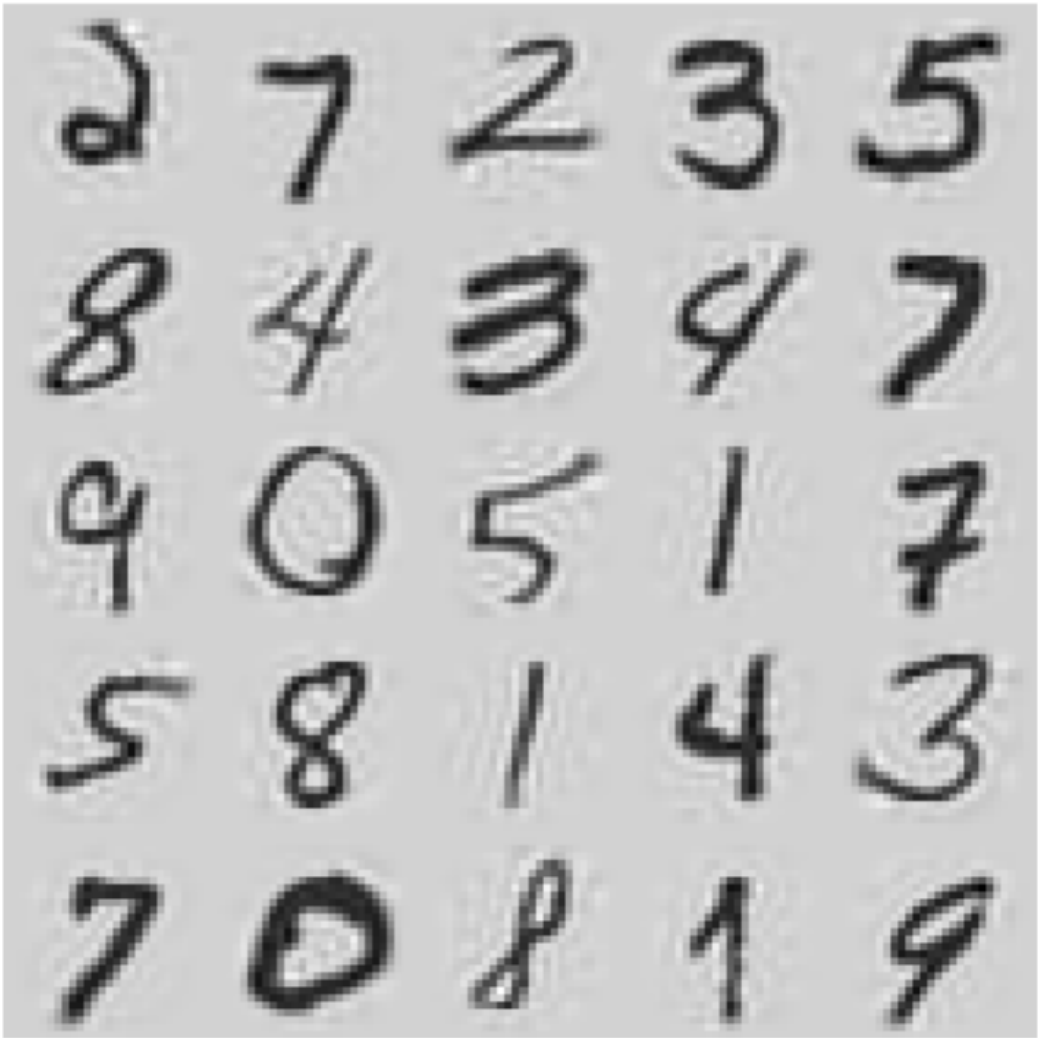
PCA

Original



784 dimensions each

Compressed



~150 dimensions each

PCA with Sklearn

- from **sklearn.decomposition** import **PCA**
- `pca = PCA(n_components=2)`
OR
`pca = PCA(n_components=0.95)`
- `X_reduced = pca.fit_transform(X_train)`
- `print(pca.n_components_)`
- `print(np.sum(pca.explained_variance_ratio))`
- `pca.inverse_transform(X_reduced)`

PCA Implementation with Sklearn

```
pcaTest = PCA(n_components=0.95)  
X_reducedTest = pcaTest.fit_transform(X_test)
```

```
X_reducedTest.shape
```

```
(17500, 153)
```

```
pcaTest.n_components_
```

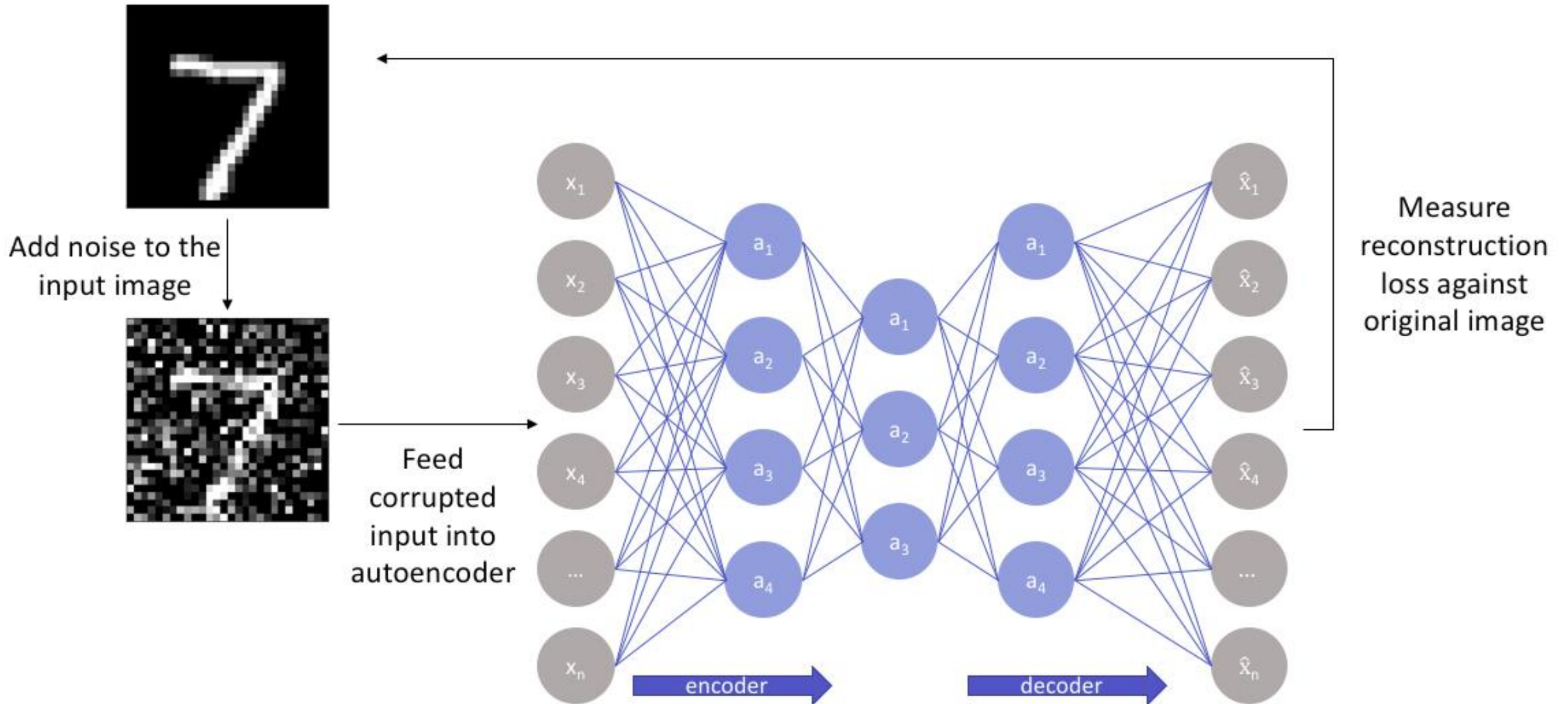
```
153
```

```
np.sum(pcaTest.explained_variance_ratio_)
```

```
0.9500923711219801
```

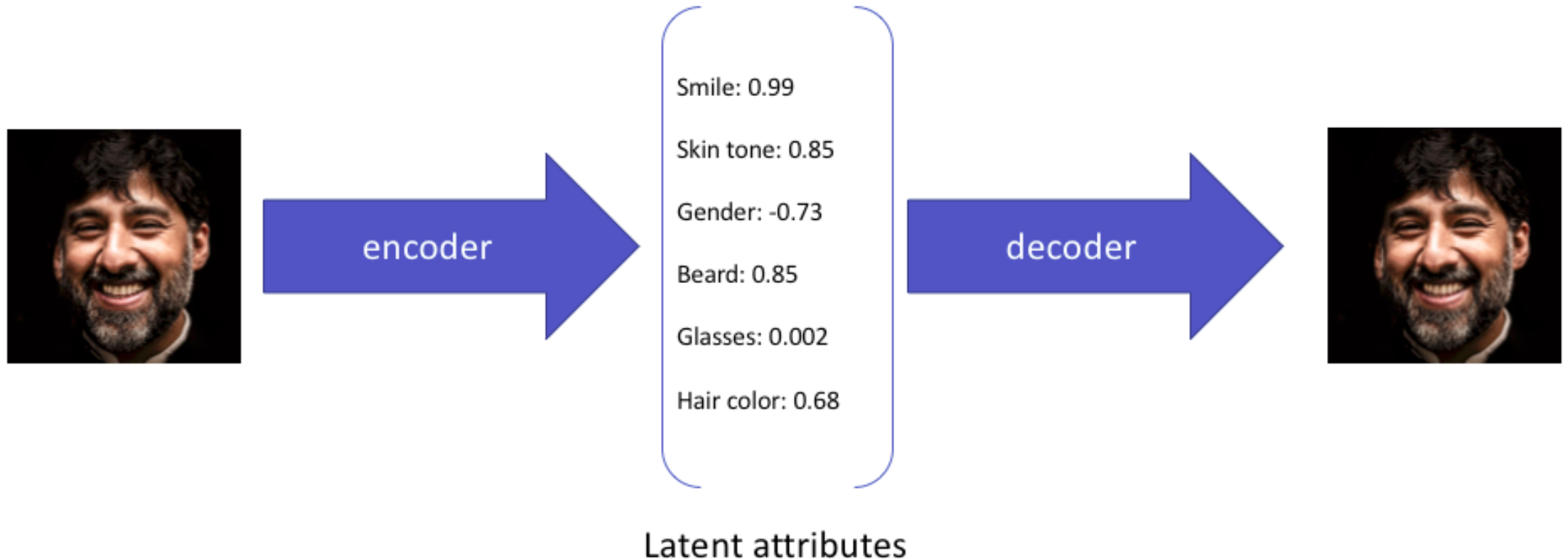


Autoencoder application: remove noise (de-noising)



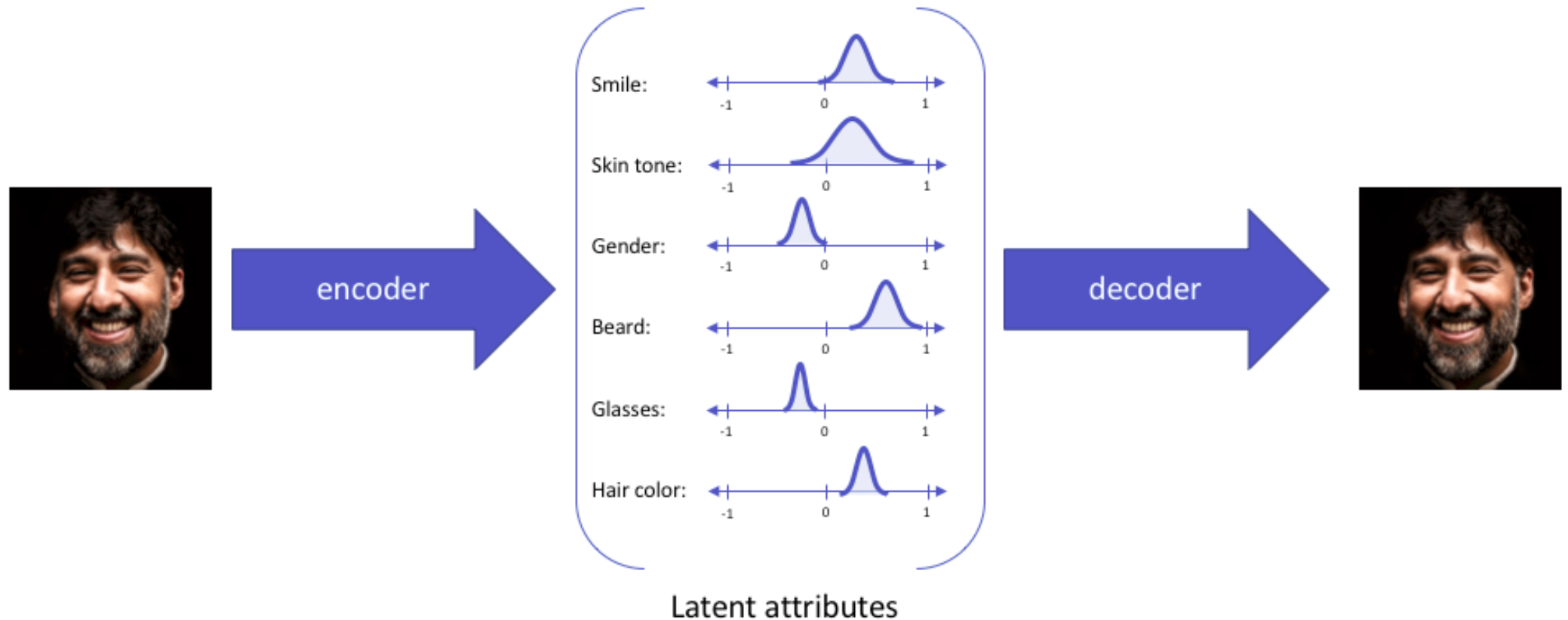
Towards Variational encoders

- While attributes explain the image and can be used in reconstructing the image from the compressed latent space, they do not allow attributes to be expressed in a probabilistic fashion



Variational autoencoders

- Variational represent latent attributes as a **probability distribution**
- The formation of a continuous latent space that can be easily sampled and interpolated.



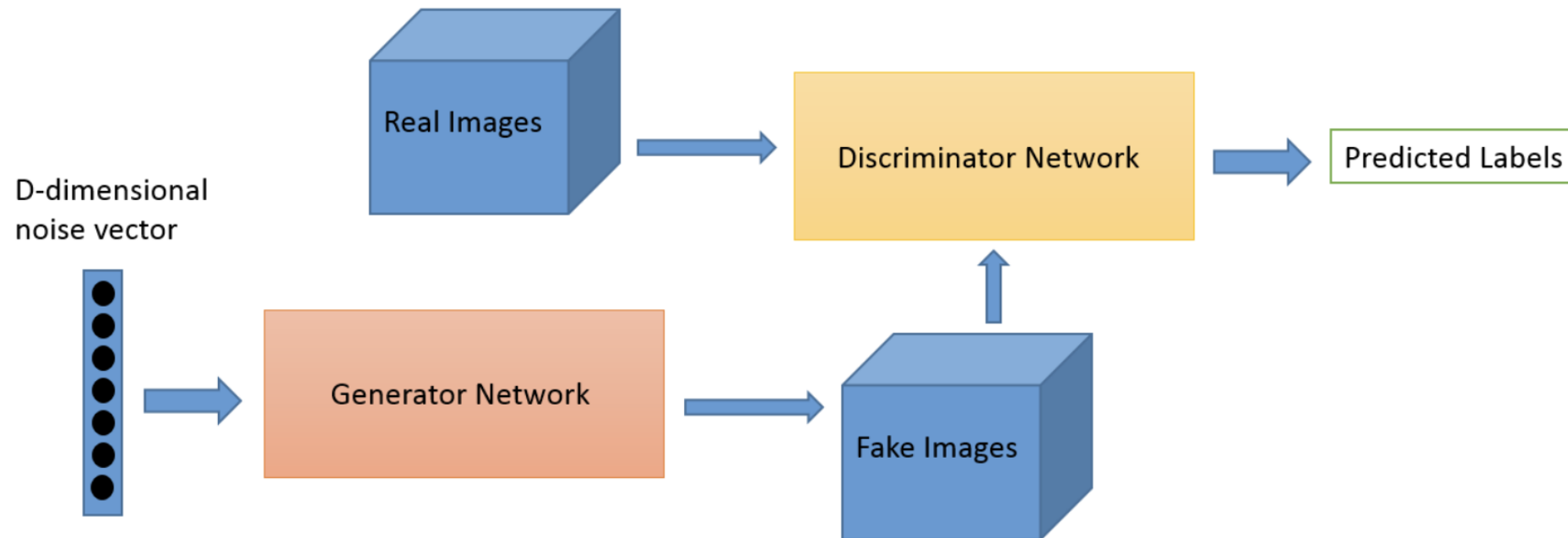
Four important Autoencoder Hyperparameters

- **Code size (bottleneck size)** decides how much the **data has to be compressed**. This can also act as a **regularisation** term.
- **Number of layers** will determine the **depth** of the **encoder** and the **decoder**. While a higher depth increases model complexity, a lower depth is faster to process.
- **Number of nodes per layer** (which defines the weights used per layer). Typically, the number of nodes decreases with each subsequent layer in the autoencoder as the input to each of these layers becomes smaller across the layers.
- **Reconstruction Loss** is highly dependent on the type of input and output we want the autoencoder to adapt to. If we are working with image data, the most popular **loss functions** for reconstruction are **MSE Loss and L1 Loss**. In case the inputs and outputs are within the range $[0,1]$, as in MNIST, we can also make use of **Binary Cross Entropy** as the reconstruction loss.

Generative Adversarial Networks (GANs)

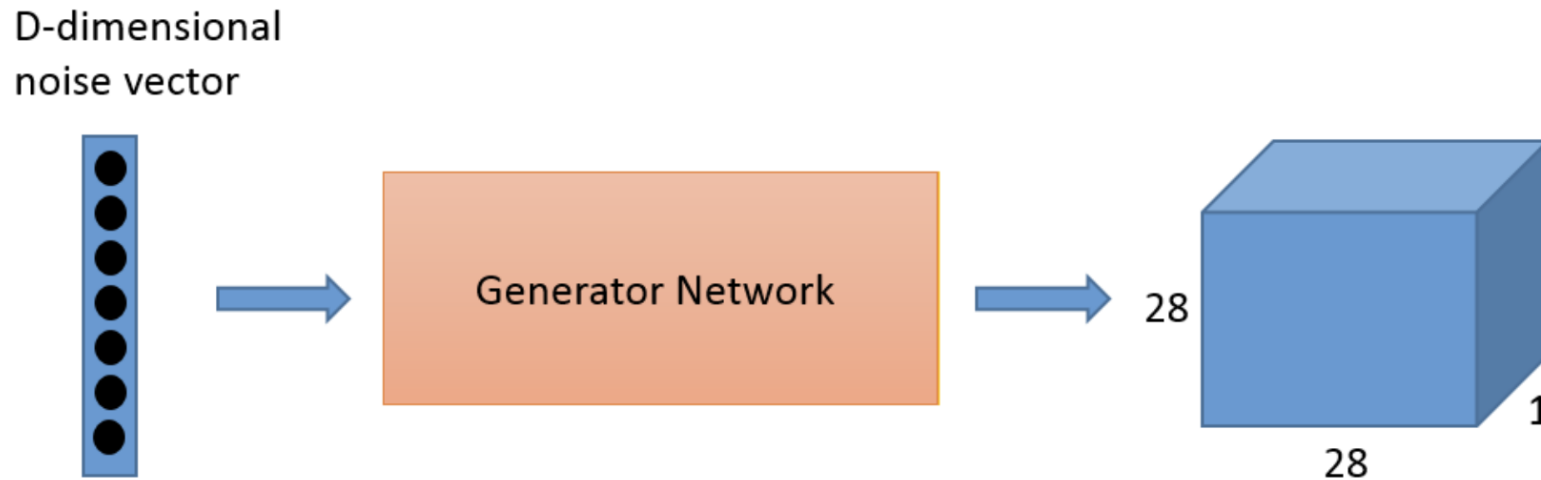
GAN architecture

- Generative Adversarial Networks consist of two models:
 - Generative model
 - Discriminative model



GAN architecture: Generator

- Generators start from noise and learn the patterns of the discriminator network (by rejection)



GAN architecture: Discriminator

- The discriminator is trained on the 'real' images
- Effective method to detect counterfeits is to study the 'real thing'



Example of GAN evolution

- Starting from noise, the two models compete against each to produce to generate more realistic images.

